

TAHIR BIN SHAMIM
10 PEARLS PAKISTAN



PROJECT REPORT

AQI Forecasting System

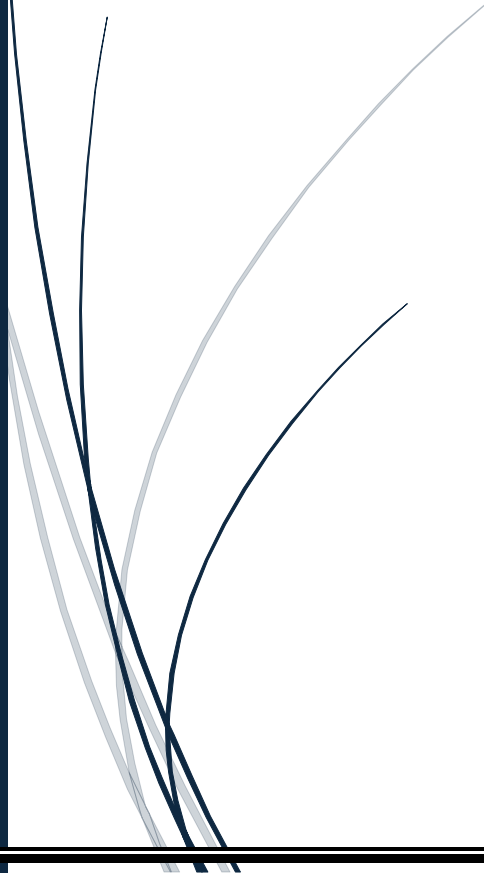


Table of Contents

1. Introduction	2
2. Project Objectives	2
3. System Architecture Overview	3
4. Feature Pipeline Development	3
4.1 Data Ingestion	3
4.2 Daily Aggregation	3
4.3 Feature Engineering	4
4.4 Feature Storage	4
5. Historical Data Backfill	5
6. Training Pipeline Implementation	5
6.1 Model Experiments	5
6.2 Model Selection: Elastic Net	6
6.3 Multi-Step Forecasting	6
7. Automated Pipeline Execution	6
8. Web Application Dashboard	7
9. Advanced Analytics	7
10. Results	8
11. Challenges Faced	8
12. Future Improvements	8
13. Conclusion	9

KARACHI AQI FORECASTING SYSTEM

END-TO-END MLOPS PIPELINE

1. Introduction

This internship project focuses on designing and implementing a complete **Air Quality Index (AQI) forecasting system** for Karachi. The objective is to build an end-to-end machine learning pipeline that automatically collects environmental data, generates features, trains predictive models, and presents real-time forecasts through an web dashboard.

The system uses AQI data from **AQICN** and integrates modern MLOps practices to ensure automation, scalability, and reproducibility.

The final outcome is a working pipeline capable of predicting AQI for the next three days and visualizing results through an interactive dashboard.

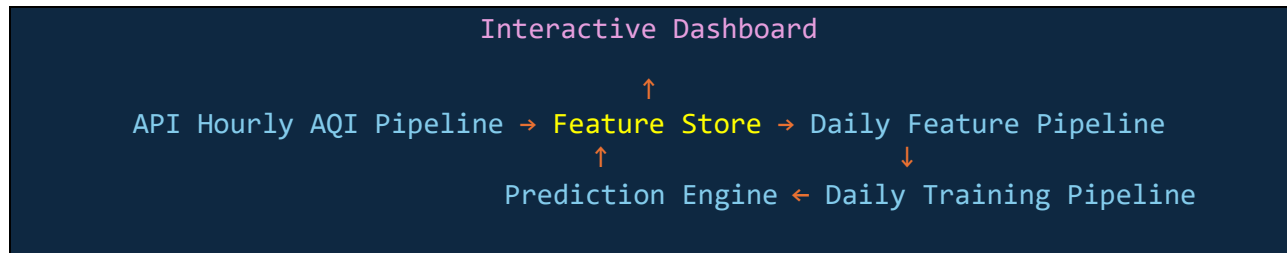
2. Project Objectives

The main objectives of the project were:

- Build an automated AQI data ingestion pipeline
- Engineer meaningful time-series features
- Develop and compare multiple ML models
- Select the best performing forecasting model
- Automate training and prediction workflows
- Create an real-time monitoring dashboard
- Apply MLOps best practices

3. System Architecture Overview

The system follows a modular architecture:



Data is stored in **MongoDB**, which acts as a lightweight feature store and model data repository.

4. Feature Pipeline Development

4.1 Data Ingestion

The system fetches hourly AQI data from the AQICN API.

Reason for selection:

- Reliable global air quality coverage
- Real-time availability
- Simple REST API integration
- Suitable for automated pipelines

Hourly ingestion ensures high temporal resolution and enables accurate daily aggregation.

4.2 Daily Aggregation

Hourly AQI values are averaged to compute daily AQI.

Reason:

- Reduces noise from short-term fluctuations
- Matches standard environmental reporting
- Provides stable training signals for ML models

4.3 Feature Engineering

The following features are generated:

Time-based features

- Day
- Month
- Day of week

Lag features

- AQI lag 1, 3, and 7 days

Rolling averages

- 3-day rolling mean
- 7-day rolling mean

Reason:

AQI is a time-series signal with strong autocorrelation. Lag and rolling features help models capture temporal dependencies and seasonal patterns.

4.4 Feature Storage

Processed features are stored in MongoDB collections:

- Raw hourly data
- Daily engineered features
- Training datastore
- Prediction results

Reason:

MongoDB provides:

- Flexible schema design
- Fast read/write performance
- Easy integration with Python

- Scalability for growing datasets

It acts as a practical feature store for internship scope.

5. Historical Data Backfill

A backfill process generates features for past timestamps to create a historical training dataset.

Reason:

Machine learning models require historical context. Backfilling ensures:

- Sufficient training samples
- Consistent feature structure
- Reproducible experiments

The system uses approximately one year of historical AQI data.

6. Training Pipeline Implementation

6.1 Model Experiments

Multiple algorithms were tested:

- Random Forest
- Gradient Boosting
- Ridge Regression
- Lasso Regression
- Elastic Net

All models were evaluated using:

- RMSE
- MAE
- R^2 score

Reason for experimentation:

Different algorithms capture patterns differently. Comparing models ensures objective performance selection.

6.2 Model Selection: Elastic Net

Elastic Net produced the best performance.

Reasons for choosing Elastic Net:

- Works well with small datasets (~365 samples)
- Combines Ridge and Lasso regularization
- Reduces overfitting
- Handles correlated lag features effectively
- Provides stable multi-step forecasting

Tree-based models tended to overfit due to limited data.

6.3 Multi-Step Forecasting

A multi-output regression model predicts:

- AQI_{t+1}
- AQI_{t+2}
- AQI_{t+3}

Reason:

Simultaneous forecasting improves efficiency and maintains temporal consistency.

7. Automated Pipeline Execution

The system is designed to support automated workflows similar to orchestration tools like **Apache Airflow**.

Pipelines include:

- Hourly feature ingestion
- Daily model retraining

- Automatic prediction storage

Reason:

Automation ensures:

- Consistent updates
- Reduced manual intervention
- Production-ready workflow structure

8. Web Application Dashboard

An interactive dashboard was built using **Streamlit**.

Dashboard Features

- Live AQI display
- Hazardous Alert
- 3-day forecast cards
- Trend visualization
- Historical data tables
- Model evaluation metrics
- Dark-mode modern UI

Reason for using Streamlit:

- Rapid development
- Native Python integration
- Interactive visualization support
- Ideal for ML dashboards

9. Advanced Analytics

The system supports:

- Exploratory trend analysis

- Forecast visualization
- Model performance tracking

These features help interpret predictions and monitor system reliability.

10. Results

The final system successfully:

- Automates AQI data collection
- Generates engineered features
- Trains forecasting models
- Produces 3-day predictions
- Displays results in real time

Elastic Net demonstrated the best predictive performance among tested models.

11. Challenges Faced

- Handling small dataset size
- Preventing model overfitting
- Maintaining time-series consistency
- Ensuring database reliability
- Managing timezone alignment

Each challenge was addressed through careful pipeline design and validation.

12. Future Improvements

Potential enhancements include:

- Larger multi-year dataset integration
- Advanced deep learning models
- Explainability tools

- Cloud deployment
- Feature store integration
- Automated hyperparameter tuning

13. Conclusion

This internship project demonstrates a complete MLOps workflow for AQI forecasting. The system integrates data engineering, machine learning, automation, and visualization into a cohesive pipeline.

The project highlights practical skills in:

- Time-series modeling
- Feature engineering
- Model evaluation
- Pipeline automation
- Dashboard development
- MLOps architecture

The final solution is scalable, extensible, and suitable as a foundation for real-world environmental monitoring systems.